https://doi.org/10.1007/s11704-025-41159-w

RESEARCH ARTICLE

Multi-instance multi-label position-aware doubly graph convolutional networks

Zhi LI, Teng ZHANG✉, Yilin WANG, Caiwu JIANG, Xuanhua SHI, Hai JIN

National Engineering Research Center for Big Data Technology and System, Services Computing Technology and System Lab, Cluster and Grid Computing Lab, School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan 430074, China

Received October 29, 2024; accepted March 10, 2025

E-mail: tengzhang@hust.edu.cn

Abstract

Multi-instance multi-label learning is a general framework in which each sample is represented as a bag of instances associated with multiple labels. However, two weaknesses remain and hinder its performance in real-world tasks. One is that bag generators often neglect positional information (e.g., the location of a pixel in the image) when generating bags, making position-related labels indistinguishable. The other is that the MIL assumption does not always hold. In some real-world tasks, labels have hierarchical low-level concepts, and these concepts are related to certain combinations of instances instead of one single instance. In this paper, we propose the Position-Aware Doubly Graph Convolutional Networks (**padGCN**). On the one hand, **padGCN** generates bags by arranging instances in a multi-instance graph to aggregate instances' features by exploiting positional relationships among them. Then instances that aggregate other instances' features are input into a neural network to obtain sub-sub-concepts used for multi-label learning. On the other hand, **padGCN** learns sub-concepts from labels and organizes sub-sub-concepts, sub-concepts, and labels in a tripartite multi-label graph in hyperbolic space to exploit their hierarchical structure. Experiments are conducted on 6 image and text data sets. Compared to the SOTA methods, padGCN averagely achieves 5% improvement on 7 measurements. Pair-wise t-test results on 42 experiments indicate that padGCN is significantly better than SOTA methods in 30 experiments, comparable to SOTA methods in 12 experiments, and never worse than SOTA methods, which verifies the superiority and robustness of padGCN. Runtime experiments show that padGCN is comparable to SOTA methods and is computationally efficient.

Keywords

multi-instance; multi-label; GCN

1 Introduction

Multi-Instance Multi-Label (MIML) [1] learning is a generalization of the single-instance single-label learning, in which each sample is represented as a bag of instances associated with multiple labels. Due to this generality, many real-world tasks can be formalized as MIML learning problems such as image classification [2–4] and text classification [5,6].

From the aspect of multi-instance learning, many methods follow the standard assumption [7] that instances are independent and identically distributed (IID) and labels are triggered by key instances. Methods like MIMLSVM [1] convert multiple instances into a single one to transform it into a multi-label learning problem. However, this standard MIL assumption cannot be satisfied in many real-world scenarios, and a new assumption that instances are not independent and the combination of instances triggers the bag label [8].

One feasible method is to use GCN to aggregate instances' features and learn such combination [9]. In recent years, instances' positions have been noticed in some scenarios for exploring their relationships and combinations. In a Drosophila gene expression pattern annotation task, instances are found associated with the position where it is located in the image [10]. Another research calculates the distances using instances' position in Mahalanobis space between bags, which improves the performances of protein function prediction tasks [11]. However, the instances' positions have not been used as an important feature. Besides, the methods above neglect the importance of instances' positions in aggregating instance features to generate instance combinations. Positional features have great influence in aspects like instance combination. Here is an example to show such importance. An image showing that the sun is near the horizon is labeled *morning* or *dawn*, and another one showing that the sun is far from the horizon is labeled *noon*. A similar phenomenon occurs in other scenarios like text classification tasks.

For the multi-label part of MIML learning, methods like MIMLBoost [1] following MIL assumption reduce one multi-label problem into multiple binary classification ones and simply combine

the predictions of each label to form a final one. Some other methods notice that there are correlations between labels and exploit such relationships between labels via their semantic information [3,12]. An existing method [13] learned label distribution through label ambiguity and utilized label correlation. MLDL [14] obtains its label distribution by utilizing the side information of multi-modal data. ALDL-kMMD [15] captures the structural information of data and labels and extracts the most representative instances from unlabeled instances. Some methods introduce intermediate spaces for label augmentation or disambiguation. For example, under the multi-instance partial-label learning framework, MIPLGP [16] transforms the candidate label set into a logarithmic space to yield the disambiguated and continuous labels via an exclusive disambiguation strategy, which is suitable for samples with only one ground-truth label. Some methods call this shared intermediate space between instances and labels sub-concepts or sub-spaces [17,18]. Sub-concepts are seen as subsets of labels and intersections of labels, which can help explore labels' relationships and alleviate the issue of the "curse of dimensionality" by decreasing the dimension of instances' label [19]. Sub-concepts of different labels are independent in these works. However, labels may have intersections, which means they may have the same sub-concepts. Besides, these works neglect the hierarchical structure of sub-sub-concepts, sub-concepts, and labels, which can be studied in a tripartite graph structure.

Motivated by the findings above, we propose Position-Aware Doubly Graph Convolutional Networks (**padGCN**) in this paper. Firstly, **padGCN** generates instances from images, texts, etc., representing them with positional and semantic features, and introducing a contrastive learning method for feature augmentation. Secondly, **padGCN** organizes instances in a graph based on their similarities, which are derived from inter-instance distances and instances' semantic features. Thirdly, it uses deep neural networks to obtain sub-sub-concepts, which can be seen as predictions in the perspective of multi-instance. Fourthly, **padGCN** organizes sub-sub-concepts, sub-concepts, and labels in a tripartite graph projected to hyperbolic space and uses a GCN to aggregate their features for relationship exploitation. And eventually, **padGCN** uses sub-sub-concepts that aggregate features of sub-concepts and labels to generate final label predictions. Our contributions are as follows:

- **padGCN** introduces an innovative bag generation method focusing on the positional features of instances that are barely investigated formerly and a contrastive learning module that other bag generators hardly use, which both enhance instance representations.
- **padGCN** innovatively introduces positional features in multi-instance relationship exploitation and organizes instances in a graph by similarities of positional and semantic features. In this way, correlated instances' features are aggregated and instance combinations are formed with a weighted GCN. Its effectiveness is verified via an ablation study.
- **padGCN** pioneers organizing sub-sub-concepts, sub-concepts, and labels in a tripartite graph in hyperbolic space, instead of thinking sub-concepts of different labels are independent. Then a GCN is used to exploit their hierarchical relationships and aggregate their features.

The rest of this paper is organized as follows. We summarize recent research in Section 2. And we propose **padGCN** in detail in Section 3. The experiments we conduct and our conclusions are stated in Sections 4 and 5.

2 Related work

For the multi-instance part, there are two main kinds of methods. One is based on instance-level features by representing a bag with the linear combinations of its instances. For example, M3GN [20] applies the attention mechanism to represent bag features as a weighted sum of instance ones. MIML-LLMC [21] applies the attention mechanism for learning bag features from different aspects. The other kind of method is based on bag-level features. For example, MIMLSVM [1] first performs a k-medoids clustering on all bags and then represents each bag with a vector, every dimension of which is the Hausdorff distance between a medoid and the bag itself. MIMLRBF [22] performs k-medoids clustering to learn typical bag features and transforms bag features with a radial basis function based on a variant of Hausdorff distance. KISAR [23] learns several instance prototypes by performing k-means clustering on all instances and represents each bag with a vector, every dimension of which is the similarity between an instance prototype and the bag. And researchers start using GNN for multi-instance learning to generate bag-level embeddings [24].

For the multi-label part, there are two main kinds of methods as well. One is to simply convert multi-label classification tasks into multiple binary ones. Typical methods are MIMLBoost [1], MIMLSVM, MIML-kNN [25], and DeepMIML [17], just to name a few. The other one is to exploit the relationships between labels. Among them are M3GN which performs GCN on labels to aggregate their features, MIMLfast [18] which learns a subspace sharing by all labels to exploit their relationships, MIML-LLMC which learns label manifolds for label correlation extraction.

Besides these kinds of methods, there are other methods whose contributions lie somewhere else as well. Such as MIML-GAN [26], which applies generative adversarial networks in MIML learning and proposes an innovative bag generator for overlapping signal waveform recognition, and M3MIML [27] which optimizes the model by maximizing the margin of bags.

3 Proposed method

In this section, we detail our **padGCN** method. First are some notations employed throughout the subsequent sections.

We begin by defining the data set as $\{(\mathcal{X}_i, y_i)\}_{i\in[N]}$, where $\mathcal{X}_i = \{x_j\}_{j\in[n_i]}$ represents a bag comprising n_i instances with $x_j \in \mathbb{R}^d$, and $y_i \in \{0,1\}^m$ denotes a label vector composed of m labels. Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ denote the undirected graph, where $\mathcal{V} = \{v_i\}_{i\in[N]}$ is the node set and $\mathcal{E} = \{(v_i, v_j)\}_{v_i, v_j \in \mathcal{V}}$ is the edge set. Let $\mathbf{A} = [a_{ij}]_{i,j\in[N]}$ denote the adjacency matrix with

$a_{ij} \in \{0,1\}$. The diagonal matrix $\mathbf{D} = [d_{ii}]_{i\in[N]}$ is the degree matrix with $d_{ii} = \sum_j a_{ij}$.

In the multi-label learning part, we project sub-sub-concepts, sub-concepts, and labels onto a Poincare ball $(\mathcal{B}^n, g_x)$ with a radius equal to 1 in the hyperbolic space, where $\mathcal{B}^n = \{x \in \mathbb{R}^n \mid \|x\| < 1\}$, $g_x = (\frac{2}{1-\|x\|^2})^2 g^E$. g^E is an Euclidean metric tensor, which is an identity matrix of n dimensions. Mobius addition $\oplus$ and Mobius multiplication $\otimes$ are used for vector and matrix calculation in hyperbolic space. In a Poincaré ball with a radius equal to 1, Mobius addition and Mobius multiplication are defined as:

$$x \oplus y := \frac{(1+2\langle x,y\rangle + \|y\|^2)x + (1-\|x\|^2)y}{1+2\langle x,y\rangle + \|x\|^2\|y\|^2}, \tag{1}$$

$$\mathbf{M} \otimes x := \tanh\left(\frac{\|\mathbf{M}x\|}{\|x\|}\tanh^{-1}\|x\|\right)\frac{\mathbf{M}x}{\|\mathbf{M}x\|}, \tag{2}$$

where $x, y \in \mathbb{D}^n, \mathbf{M} : \mathbb{R}^n \mapsto \mathbb{R}^m$.

3.1 Bag generation

Our approach highly focuses on instances' positional features and inter-instance distances. The positional features of an instance represent its spatial location within the bag, such as pixel coordinates in images or sentence sequences in texts. Inter-instance distances are computed from the positional features in Euclidean space. The core of the bag generation process of **padGCN** is capturing the positional features of each instance within its feature vector. We will clarify how this process unfolds for both image and text data sets. Subsequently, we introduce a contrastive learning step for feature augmentation.

Position-aware feature extractions. In our method, the elements in each bag, such as the pixels in images and sentences in texts, are represented by both their positional features and semantic features. Our method clusters them into different clusters and each cluster is seen as an instance, and the instance's feature is the feature of the corresponding cluster center, i.e., the weighted averaging feature of all elements in the cluster.

For image data, we see pixels as the elements in images. Our approach clusters contiguous pixels with similar colors into clusters, treating each cluster as an instance. Each pixel's semantic features are its RGB values, while its positional features are its two-dimensional coordinates in the image. Positional features are important during clustering, ensuring that our method will not cluster pixels with significant distances but high color similarity into the same cluster. Our method concatenates positional and semantic features to obtain a final feature vector for each instance. Following this, **padGCN** performs clustering via self-organizing map (SOM) neural networks [28] on the pixels, where each cluster center is treated as an instance. Figure 1 shows an example.

For example, consider an instance with the features $(54, 12, 255, 78, 124)$. This means the spatial coordinates of the corresponding cluster center are $(54, 12)$, and the RGB values of the corresponding cluster center are $(255, 78, 124)$.

To show the scalability of our method, we combine it with an existing bag generator for text data [1,29]. We segment text into portions using overlapping windows. Each of these segments is regarded as an instance. Our method excludes function words and employs BERT [30] for word embedding. Besides, we assign greater importance to words with higher frequencies based on term frequency. This involves calculating a weighted average for each bag of words, generating its semantic feature. The positional features are one-dimensional attributes representing the instance's sequence in the text. For a text with n segments, these positional features range from 1 to n.

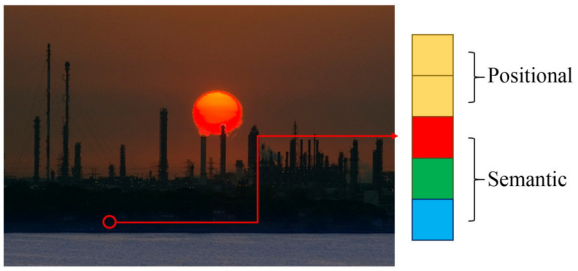

Fig. 1 Feature vector for pixels consists of both their positional and semantic features. Semantic features are pixels' RGB values. Positional features are pixels' coordinates

Representation enhancement by contrastive learning. Inspired by methods like SimCLR [31] and GCLF [32], the contrastive learning component of our method aims to maximize the similarity between an instance and its projection while maximizing the differences between the projections of the other instances and it.

Many existing contrastive learning methods aim to maximize the similarity among positive instances while maximizing the differences between negative ones. A direct approach to introduce the contrastive learning component into our method is maximizing the similarity between key instances and minimizing the similarity between key instances and other non-key instances. However, identifying these key instances in real-world data sets is difficult. Therefore, an acceptable alternative is to maximize the similarity between an instance and its own projection, which is precisely the approach adopted by our method.

padGCN employs a fully-connected neural network to project instances into high-dimension space. For a single instance, we generate features z_1 and z_2 from the instance embedding h_1. z_1 typically represents high-dimensional features obtained by using zero padding, while z_2 is obtained through projection by a fully-connected neural network. We calculate cosine similarity $s_c(z_1, z_2)$ as defined by $s_c(z_1, z_2) = \frac{z_1^\top z_2}{\|z_1\|_2\|z_2\|_2}$. For a total of N instances, the loss between z_i and z_j is defined as:

$$\ell(z_i, z_j) = -\log\frac{\exp(s_c(z_i, z_j)/\tau)}{\sum_{k=1}^{2N}\mathbb{I}(k \neq j)\exp(s_c(z_i, z_k)/\tau)}, \tag{3}$$

where $\mathbb{I}$ is indicator function, τ represents the temperature parameter, and $2N$ denotes the total number of projections. Consequently, the loss function for the contrastive learning component is

$$\mathcal{L}_1 = \frac{1}{2}\sum_{k=1}^{N}[\ell(z_{2k-1}, z_{2k}) + \ell(z_{2k}, z_{2k-1})], \tag{4}$$

where $2k-1$ and $2k$ represent the two different projections derived

from the kth instance through the projectors. By optimizing this loss function, we train a projector for representation augmentation and obtain projected instance features for the following learning.

Notably, the essence of the bag generation process in **padGCN** is to generate instances with positional features. So most bag generators can be combined with **padGCN** as long as they incorporate instances' positional features.

3.2 Multi-instance GCN with inter-instance distances

In this subsection, we explain the organization of instances within a graph structure and the role of positional features in our **padGCN** method. As stated in pervious work, attention weights suffer from undesired smoothness [33]. So our method use weights based on nodes' positional and semantic similarity rather than attention machanism.

Adjacency Based on positional and semantic Similarity. Some existing methods assess the existence of an edge between two instances only based on their semantic features. However, in real-world scenarios, changes in the inter-instance distances can lead to changes in the bag's labels. Therefore, instances influence each other not only by their semantic features but also by their positional features.

Our method computes inter-instance distances using positional features. Given two instances $x_i = s_i \| p_i$ and $x_j = s_j \| p_j$, where $\|$ represents vector concatenation and s and p are instance's positional and semantic features respectively, the inter-instance distance in Euclidean space is $\|p_i - p_j\|_2$. Following feature extraction during the bag generation step, each instance is treated as an individual node. Our method clusters all nodes within a bag into different clusters, and subsequently, all nodes within the same cluster are fully connected. Figure 2 illustrates the organization of instances into a graph structure by **padGCN**.

Weighted GCN based on inter-instance distances. Following the creation of an instance graph for each bag, our method employs a weighted Graph Convolutional Network (GCN), where the similarity matrix is determined based on both positional and semantic similarity. This similarity matrix enables our method to assign different importance to different neighbors, similar to the attention mechanism utilized in Graph Attention Networks (GAT) and EA-WGCN [34]. However, there is a notable distinction in our method. Our method considers both positional and semantic information, whereas existing methods often focus on nodes with similar semantic features, even when they are spatially distant.

It is essential to highlight that this difference leads to a unique characteristic of **padGCN**—the combinations of instances generated by our method can be seen as concrete objects, while instances generated by other methods tend to represent abstract categories of objects. The layer-wise propagation rule of the multi-instance GCN can be defined as follows:

$$\mathbf{H}^{(i+1)} = \mathrm{ReLU}(\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{S}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{H}^{(i)}\mathbf{W}^{(i)}), \tag{5}$$

where $\mathbf{H}^{(i)}$ represents the output of layer i. The $\mathbf{A} = \tilde{\mathbf{A}} + \mathbf{I}_N$ matrix is the adjacency matrix, with the addition of a diagonal identity matrix to reserve the node's own features as self-loop. $\tilde{\mathbf{D}}$ denotes the degree matrix, and $\mathbf{W}^{(i)}$ is the learnable weight matrix. Additionally, $\tilde{\mathbf{S}}$ stands for the similarity matrix, with learnable parameters. The calculation of the similarity matrix $\tilde{\mathbf{S}}$ considers both positional and semantic similarities between nodes. The elements within the unnormalized similarity matrix $\mathbf{S}$ are defined as:

$$\mathbf{S}_{ij} = \begin{cases} s(x_i, x_j), & \text{if } x_i \text{ and } x_j \text{ adjacent,} \\ 0, & \text{otherwise,} \end{cases} \tag{6}$$

where $\mathbf{S}_{ij}$ is the weight of x_i when updating the features of x_j.

Our method incorporates both positional and semantic similarities, rather than relying solely on semantic similarities. It sums up the two kinds of similarities and subsequently calculates the weighted harmonic mean for each of them. This approach alleviates the influence of extreme similarity values in just one aspect, promoting a more balanced consideration of both positional and semantic information.

Similarity design. Inspired by the F_1 score [35], our similarity function $s(x_i, x_j)$ follows a similar framework. The F_1 score is renowned for its ability to obtain a balance between precision and recall, ensuring that one measure does not dominate the other. This design aligns with our method's emphasis on both positional and semantic features, emphasizing their equal importance. We define the similarity between two nodes, $s(x_i, x_j)$, as follows:

$$s(x_i, x_j) = \frac{s_p(x_i, x_j) s_s(x_i, x_j)}{\alpha s_p(x_i, x_j) + (1-\alpha) s_s(x_i, x_j)}, \tag{7}$$

where $s_p(x_i, x_j) = 1/\|p_i - p_j\|_2$ and $s_s(x_i, x_j) = 1/\|s_i - s_j\|_2$. $s_p(x_i, x_j)$ and $s_s(x_i, x_j)$ correspond to positional and semantic similarities respectively. To enhance the adaptability of our method, we introduce the parameter α in Eq. (7) as a learnable parameter during training. Equation (7) can be rewritten as:

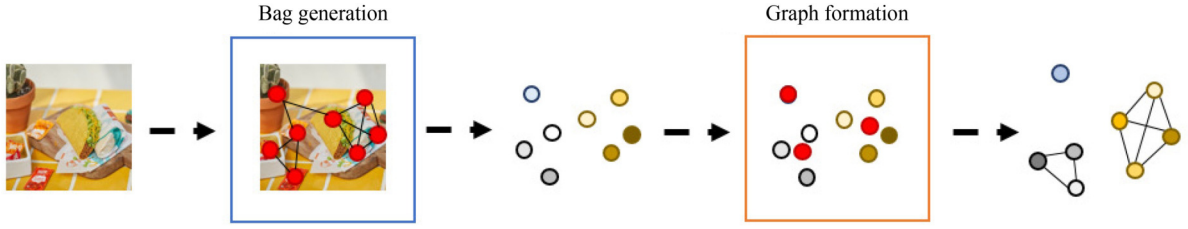

Fig. 2 An example shows the process of creating a graph from a given image. Initially, pixels are grouped into 8 clusters, with each cluster representing an instance. Subsequently, these 8 instances are further clustered into 3 clusters, and instances within the same cluster are regarded as correlated and are connected in the graph

$$\frac{\|p_i - p_j\|_2 \|s_i - s_j\|_2}{\alpha\|p_i - p_j\|_2 + (1-\alpha)\|s_i - s_j\|_2}. \tag{8}$$

Our method leverages these components of similarity by calculating the multiplicative inverse of their Euclidean distances. The larger the Euclidean distances, the less similar the nodes are in either semantic or positional features, resulting in lower values for $s_s(x_i, x_j)$ or $s_p(x_i, x_j)$. Notably, $\|p_i - p_j\|_2$ is always positive, and $\|s_i - s_j\|_2$ is non-negative, ensuring that division by zero does not occur.

3.3 Multi-Label GCN

Following feature transformation and L latent layers, our method derives sub-sub-concepts $\mathbf{O}$ through $\mathbf{O} = \mathbf{W}\mathbf{H}^{(L)}$. So far, our method only learns from a multi-instance perspective. In this subsection, we'll explain how our method learns from the multi-label perspective and exploits the relationships between sub-sub-concepts, sub-concepts, and labels.

In **padGCN**, we posit that labels interact through latent sub-concepts. For example, consider labels like "sky" and "sea" sharing the common sub-concept "horizon" implicitly. In turn, the sub-concept "horizon" might have sub-sub-concepts, such as "black boundary", which is a concrete object learned from the multi-instance portion of our method. By learning sub-concepts from sub-sub-concepts and learning labels from sub-concepts, our approach gains the capability to increase the confidence of a label when a related label exists or not. To explore these relationships, **padGCN** learns labels from sub-concepts and sub-concepts from sub-sub-concepts and employs tripartite graphs to organize these elements along with labels. Figure 3 shows an overview of the multi-label learning part of **padGCN**.

Firstly, similar to [36], **padGCN** projects sub-sub-concepts $\mathbf{O}$, sub-concepts $\mathbf{C}$, and labels $\mathbf{L}$ into hyperbolic space with $f(x) = \text{arctanh}(\|x\|)\frac{x}{\|x\|}$. In this step, the original features of sub-sub-concepts are the outputs of the multi-instance part of **padGCN**. Features of sub-concepts are initialized as all-zero vectors. Features of labels are initialized as their word embeddings. Subsequently, sub-sub-concepts, sub-concepts, and labels are updated by

$$[\hat{\mathbf{O}}; \hat{\mathbf{C}}; \hat{\mathbf{L}}] = \sigma(\exp_{x'}(\mathbf{M}\log_{x'}[\mathbf{O}; \mathbf{C}; \mathbf{L}])), \tag{9}$$

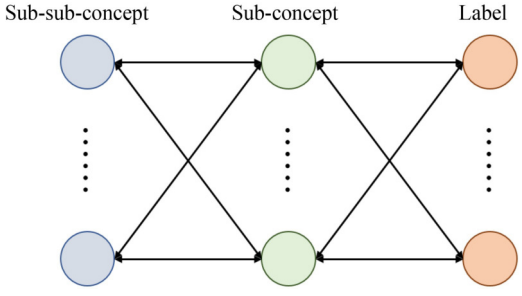

Fig. 3 Propagation process within the multi-label GCN component of **padGCN** involves learning sub-concepts from the sub-sub-concepts generated by the multi-instance section of **padGCN**. Subsequently, the final label predictions are derived from a feature aggregation of sub-sub-concepts, sub-concepts, and labels

where $\sigma(\cdot)$ is Sigmoid activation function. Exponential mapping $\exp_{x'}(v)$ and logarithmic mapping $\log_{x'}(y)$ are defined respectively as:

$$\exp_{x'}(v) = x \oplus \left(\frac{v}{\|v\|}\tanh\frac{\|v\|}{2}\right), \tag{10}$$

$$\log_{x'}(y) = \text{arctanh}(\|-x' \oplus y\|)\frac{-x' \oplus y}{\|-x' \oplus y\|}, \tag{11}$$

where x' is a fixed point in the Euclidean space, $\mathbf{M} \in \mathbb{R}^{(s+c+l)\times(s+c+l)}$ is a learnable convolutional parameter matrix, and s, c, and l are the numbers of sub-sub-concepts, sub-concepts, and labels respectively. The matrix $\mathbf{M}$ is normalized, originating from $\tilde{\mathbf{M}}$. Specifically, $\tilde{\mathbf{M}} = [c_{ij}]_{i,j\in[s+c+l]}$, with each c_{ij} denoting a learnable parameter measuring the relationship between sub-sub-concepts, sub-concepts, and labels. The output $[\hat{\mathbf{O}}; \hat{\mathbf{C}}; \hat{\mathbf{L}}]$ represents the features of sub-sub-concepts, sub-concepts, and labels after their feature aggregation.

Subsequently, **padGCN** inputs $\hat{\mathbf{O}}$, which are features aggregated from sub-sub-concepts, sub-concepts, and labels, into the classifier. **padGCN** obtains the output of the multi-label GCN, which is as follows:

$$\hat{y} = [\hat{y}^1, \hat{y}^2, \ldots, \hat{y}^m],\ \hat{y}^i = \max\{\sigma(\mathbf{M}^i\hat{\mathbf{O}})_{i,:}\}, \tag{12}$$

where $\mathbf{M}^i$ is the parameter matrix of the trainable classier, which is a neural network, for label i.

Then we employ the binary cross-entropy loss, defined as follows:

$$\mathcal{L}_2 = -\sum_{i=1}^{m} y^i \log(\sigma(\hat{y}^i)) + (1-y^i)\log(1-\sigma(\hat{y}^i)), \tag{13}$$

where y^i and $\hat{y}^i$ represent the true value and prediction of label i respectively. The final loss function that **padGCN** optimizes is defined as a weighted average of $\mathcal{L}_1$ and $\mathcal{L}_2$ as follows:

$$\mathcal{L} = \beta\mathcal{L}_1 + (1-\beta)\mathcal{L}_2, \tag{14}$$

where we employ Adam [37] as the optimizer. The pseudo-code of **padGCN** is shown in Algorithm 1.

4 Experiment

In this section, we present the results of our experiments, which aim to validate the effectiveness of our proposed bag generation method and weighted GCN design. We perform experiments on 4 image data sets and 2 text data sets. In this section, we demonstrate the effectiveness of **padGCN** in both aspects and provide evidence that **padGCN** holds a distinct advantage. To underscore this advantage, an ablation study is also performed, in which we removed positional features and employed non-weighted GCN as the baseline method.

4.1 Setting

● Data sets

In this subsection, we provide an overview of the data sets used in our experiments for image and text classification tasks. We utilized a diverse set of 6 public real-world data sets, including Scene [38],

Algorithm 1 padGCN

Require: Raw data with A elements (e.g., pixels, words, etc.), L layers of instance GCN
Ensure: Labels $\hat{\boldsymbol{y}} = \{\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_m\}$

for $a = 1$ to A **do** ▷ Extract features
    $\text{feature}_{(a)} = (\text{Position}, \text{Semantic})$
end for
Cluster to generate n instances ▷ Bag generation
Organize n instances in a graph structure
Calculate instance similarity matrix $\mathbf{S}$
for $l = 1$ to L **do** ▷ Graph convolution layers
    $\mathbf{H}^{(l+1)} = \text{ReLU}(\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{S}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{H}^{(l)}\mathbf{W}^{(l)})$
end for
Calculate sub-sub-concept $\mathbf{O} = \mathbf{W}\mathbf{H}^{L}$
Calculate $[\hat{\mathbf{O}}; \hat{\mathbf{C}}; \hat{\mathbf{L}}] = \sigma(\exp_{x'}(\mathbf{M}\log_{x'}[\mathbf{O}; \mathbf{C}; \mathbf{L}]))$ ▷ Multi-label GCN layers
Calculate final outputs $\hat{\boldsymbol{y}} = [\hat{y}^1, \hat{y}^2, \ldots, \hat{y}^m]$ where $\hat{y}^i = \max\{\sigma(\mathbf{M}^i\hat{\mathbf{O}})_{i,:}\}$ ▷ Label prediction
return $\hat{\boldsymbol{y}}$ ▷ Return predicted labels

MIML-TEXT [39], corel5k [40], DeliciousMIL[1)], FLICKR25K [41], and MS-COCO [42]. A brief summary of these data sets is provided in the Table 1. The detailed information of each data set is as follows:

Scene contains 2,000 natural scene images categorized into 5 labels: desert, mountains, sea, sunset, and trees. Approximately 20% of the images have more than one label, resulting in an average of about 1.24 labels per bag.

MIML-TEXT is derived from the Reuters-21578 collection, which comprises 2,000 bags, each with varying numbers of instances. Instances are represented as 243-dimensional feature vectors, associated with 7 possible labels.

corel5k covers 263 diverse labels, spanning categories such as dance, African, and more, encompassing images of people, natural scenes, and various subjects, consisting of 5,000 images.

DeliciousMIL contains 12,232 documents extracted from the DeliciousT140 data set. It includes a total of 20 labels, with an average of 2.90 labels per bag. Labels include categories like programming and English.

FLICKR25K assigns, comprising 25,000 images from the Flickr website, a bag of labels to each image, encompassing a total of 24 labels. Most images have fewer than 15 labels, covering diverse categories from environment-related labels like clouds, indoor, and lakes to object-related labels like animals, baby, and plant.

MS-COCO is an extensive data set that contains 117,266 training images and 4,952 validation images, annotated with 80 labels. MS-COCO supports various computer vision tasks, including object detection, segmentation, captioning, and image classification. Each data set was partitioned into training (70%), validation (10%), and test (20%) sets to facilitate our experimental evaluation.

Table 1 Data sets overview: #Bag and #Label represent the number of bags and total potential labels, respectively, and avg.#Label is the average number of labels per bag

Task	Data set	#Bag	#Label	avg.#Label
Image	Scene	2,000	5	1.24
	corel5k	5,000	263	4.25
	FLICKR25K	25,000	24	3.72
	MS-COCO	117,266	80	3.44
Text	MIML-TEXT	2,000	7	3.56
	DeliciousMIL	12,232	20	2.90

● Baseline methods

In our experiments, we evaluated the performance of **padGCN** against several baseline methods, each offering unique approaches to address MIML learning. Here's an overview of these baseline methods:

MIMLBoost transforms MIML problems into multi-instance single-label problems and employs MIboosting [43] to solve them with boosting algorithms [1].

MIMLSVM transforms MIML problems into single-instance multi-label ones and utilizes SVM as classifiers [1]. This approach effectively aggregates a bag of instances into a single instance.

MIMLfast projects instances into a shared subspace, facilitating the learning of sub-concepts to explore relationships between similar instances and labels [18].

DeepMIML leverages deep neural networks and introduces sub-concept layers for instances [17]. These sub-concept layers aim to uncover hidden connections between instances and labels.

M3GN is a multi-modal method for MIML learning [20]. It employs a GAT for instance aggregation, focusing on attention-based mechanisms.

MIML-GAN is a generative-adversarial approach designed for MIML learning in automatic waveform recognition [26]. It involves a generator creating fake representations of image and text bags and a discriminator making predictions.

MIML-LLMC transforms each bag into multiple fusion vectors, serving as label predictions [21]. It explores label correlation in a label manifold, partitioning it into groups using k-medoids.

[1)] See doi.org/10.24432/C5DK74 website.

npGCN (Non-Position Graph Convolutional Networks) represents a simplified version of **padGCN**. It removes the positional features of instances and employs a basic GCN for feature propagation.

esGCN (Euclidean Space Graph Convolutional Networks) is another simplified version of **padGCN**. It doesn't project label features or organize labels and sub-concepts in hyperbolic space. Its multi-label GCN is operated in Euclidean space.

● Metrics

We take hamming loss (HL), one error (OE), coverage error (CE), ranking loss (RL), average precision (AP), average recall (AR), and *average* F_1 (AF_1) as the performance evaluation metrics.

● Hyperparameters

For text data sets, we use BERT [30] to obtain semantic features and conduct clustering to obtain instances. For data sets Scene and corel5k, we use pixels' RGB values as semantic features and conduct clustering to obtain instances. In the initial clustering step generating instances, the cluster number is selected from the set $\{4,9,16\}$. For data sets FLICKR25K and MS-COCO, we use Faster R-CNN [44] to obtain instances and their semantic features.

In the subsequent clustering step determining whether two instances have relationships and creating instance graphs, the cluster number is selected from the set $\{3,6,12\}$. The numbers of latent layers and the latent dimension of all methods that employ neural networks are selected from the sets $\{2,3,4\}$ and $\{128,256,512\}$ respectively. The size of overlapping windows is set to 5 which is the same as it in [1]. The depth of instance graph convolutional networks are selected from the sets $\{2,3,4\}$. The temperature parameter τ is selected from the set $\{0.1,0.5,1\}$. We configure the number of sub-concepts to match the number of labels in the corresponding data set. Batch size, learning rate, and the number of epochs are selected from the sets $\{500,2000,20,000\}$, $\{0.1,0.01,0.001\}$, and $\{50,100,200\}$ respectively. β in loss function $\mathcal{L}=\beta\mathcal{L}_1+(1-\beta)\mathcal{L}_2$ is set 0.5. Other baseline method parameters are set as the same setup in their previous works.

4.2 Results

The results are summarized in Tables 2–7 based on 10 runs. Across

Table 2 Performance of **padGCN** compared to nine baseline methods on Scene data set

Metrics	HL ↓	OE ↓	CE ↓	RL ↓	AP ↑	AR ↑	AF_1 ↑	w/t/l
padGCN	**0.111±0.023**	**0.156±0.053**	**0.536±0.119**	**0.073±0.029**	**0.903±0.034**	**0.766±0.058**	**0.828±0.048**	
MIMLBoost	0.193±0.007●	0.347±0.019●	0.984±0.049●	0.178±0.011●	0.779±0.012●	0.433±0.027●	0.556±0.023●	6/0/0
MIMLSVM	0.189±0.009●	0.354±0.022●	1.087±0.047●	0.201±0.011●	0.765±0.013●	0.556±0.020●	0.644±0.018●	6/0/0
MIMLfast	0.188±0.009●	0.351±0.023●	0.207±0.012	0.189±0.014●	0.770±0.015●	0.477±0.273●	0.607±0.142●	5/1/0
DeepMIML	0.137±0.038●	0.287±0.120●	0.969±0.290●	0.181±0.074●	0.807±0.079●	0.536±0.181●	0.633±0.164●	6/0/0
M3GN	0.140±0.011●	0.215±0.025●	0.669±0.064●	0.102±0.014●	0.865±0.015●	0.670±0.036●	0.754±0.028●	6/0/0
MIML-GAN	0.253±0.075●	0.417±0.079●	1.195±0.249●	0.228±0.061●	0.556±0.069●	0.663±0.074●	0.587±0.049●	6/0/0
MIML-LLMC	0.250±0.005●	0.717±0.024●	0.730±0.010●	0.477±0.014●	0.503±0.016●	0.497±0.018●	0.503±0.016●	6/0/0
npGCN	0.135±0.013●	0.234±0.038●	0.825±0.113●	0.137±0.026●	0.864±0.026●	0.608±0.045●	0.734±0.036●	6/0/0
esGCN	0.147±0.028●	0.265±0.079●	1.038±0.127●	0.133±0.035●	0.889±0.032●	0.669±0.166●	0.796±0.052●	6/0/0

Table 3 Performance of **padGCN** compared to nine baseline methods on MIML-TEXT data set

Metrics	HL ↓	OE ↓	CE ↓	RL ↓	AP ↑	AR ↑	AF_1 ↑	w/t/l
padGCN	**0.011±0.012**	**0.025±0.016**	**0.186±0.051**	**0.005±0.003**	**0.963±0.042**	**0.967±0.039**	**0.965±0.041**	
MIMLBoost	0.160±0.010●	0.573±0.290●	2.153±0.018●	0.867±0.005●	0.407±0.004●	0.132±0.005●	0.200±0.006●	6/0/0
MIMLSVM	0.199±0.001●	0.487±0.003●	0.720±0.005●	0.222±0.002●	0.706±0.002●	0.507±0.003●	0.590±0.003●	6/0/0
MIMLfast	0.028±0.004●	0.044±0.008●	0.035±0.004	0.014±0.004●	0.972±0.005	0.500±0.276●	0.751±0.150●	4/2/0
DeepMIML	0.160±0.066●	0.231±0.218●	0.890±0.626	0.157±0.102●	0.693±0.251●	0.831±0.119●	0.717±0.206●	5/1/0
M3GN	0.070±0.030●	0.127±0.106●	0.495±0.603●	0.056±0.018●	0.815±0.181●	0.837±0.135●	0.817±0.166●	6/0/0
MIML-GAN	0.041±0.040●	0.091±0.084●	0.374±0.332●	0.037±0.015●	0.845±0.129●	0.839±0.130●	0.841±0.130●	6/0/0
MIML-LLMC	0.053±0.002●	0.092±0.006●	0.480±0.001●	0.028±0.001●	0.745±0.003●	0.721±0.031●	0.722±0.013●	6/0/0
npGCN	0.064±0.019●	0.082±0.046●	0.212±0.102●	0.107±0.027●	0.857±0.034●	0.632±0.029●	0.744±0.030●	6/0/0
esGCN	0.036±0.017●	0.081±0.026●	0.238±0.076●	0.065±0.013●	0.906±0.040●	0.912±0.019●	0.909±0.029●	6/0/0

Table 4 Performance of **padGCN** compared to nine baseline methods on corel5k data set

Metrics	HL ↓	OE ↓	CE ↓	RL ↓	AP ↑	AR ↑	AF_1 ↑	w/t/l
padGCN	**0.003±0.002**	**0.055±0.010**	**6.612±4.114**	**0.007±0.026**	**0.782±0.194**	**0.821±0.193**	**0.800±0.193**	
MIMLBoost	N/A	N/A	N/A	N/A	N/A	N/A	N/A	
MIMLSVM	0.160±0.001•	0.364±0.137•	12.153±0.018•	0.867±0.005•	0.407±0.004•	0.132±0.005•	0.200±0.006•	6/0/0
MIMLfast	0.194±0.001•	0.129±0.096•	216.769±11.158•	0.705±0.043•	0.022±0.001•	0.485±0.111•	0.041±0.003•	6/0/0
DeepMIML	0.005±0.002	0.132±0.219•	14.041±24.011•	0.021±0.045•	0.657±0.276•	0.699±0.287•	0.677±0.282•	5/1/0
M3GN	0.010±0.002•	0.063±0.038•	7.067±14.282•	0.008±0.027	0.750±0.193	0.795±0.191•	0.771±0.192•	4/2/0
MIML-GAN	0.036±0.013•	0.618±0.121•	70.611±38.206•	0.131±0.103•	0.077±0.029•	0.098±0.084•	0.077±0.028•	6/0/0
MIML-LLMC	0.013±0.000•	0.911±0.040•	7.400±0.260•	0.459±0.029•	0.069±0.019•	0.050±0.007•	0.061±0.010•	6/0/0
npGCN	0.008±0.004•	0.060±0.007•	13.267±7.700•	0.038±0.012•	0.692±0.293•	0.740±0.317•	0.715±0.304•	6/0/0
esGCN	0.010±0.001•	0.062±0.027•	13.524±8.742•	0.029±0.008•	0.680±0.019•	0.778±0.021•	0.739±0.020•	6/0/0

Table 5 Performance of **padGCN** compared to nine baseline methods on DeliciousMIL data set

Metrics	HL ↓	OE ↓	CE ↓	RL ↓	AP ↑	AR ↑	AF_1 ↑	w/t/l
padGCN	**0.107±0.018**	**0.252±0.035**	**10.073±1.231**	**0.064±0.004**	**0.731±0.016**	**0.720±0.026**	**0.724±0.007**	
MIMLBoost	N/A	N/A	N/A	N/A	N/A	N/A	N/A	
MIMLSVM	0.749±0.205•	0.487±0.120•	15.428±0.452•	0.242±0.001•	0.442±0.064•	0.546±0.078•	0.465±0.034•	6/0/0
MIMLfast	0.297±0.011•	0.429±0.151•	16.769±2.456•	0.107±0.013•	0.562±0.030•	0.587±0.030•	0.574±0.029•	6/0/0
DeepMIML	0.237±0.054•	0.332±0.219•	11.345±1.421	0.134±0.022•	0.669±0.019•	0.679±0.049•	0.673±0.029•	5/1/0
M3GN	0.150±0.003•	0.273±0.053•	10.069±3.753	0.063±0.010	0.714±0.023	0.679±0.023•	0.696±0.016•	4/2/0
MIML-GAN	0.363±0.050•	0.399±0.003•	18.294±5.123•	0.103±0.004•	0.658±0.028•	0.620±0.023•	0.637±0.015•	6/0/0
MIML-LLMC	N/A	N/A	N/A	N/A	N/A	N/A	N/A	
npGCN	0.133±0.015•	0.301±0.100•	13.417±3.418•	0.083±0.001•	0.629±0.024•	0.609±0.016•	0.618±0.018•	6/0/0
esGCN	0.154±0.022•	0.275±0.046•	12.753±6.441•	0.102±0.012•	0.653±0.037•	0.627±0.023•	0.639±0.018•	6/0/0

Table 6 Performance of **padGCN** compared to nine baseline methods on FLICKR25K data set

Metrics	HL ↓	OE ↓	CE ↓	RL ↓	AP ↑	AR ↑	AF_1 ↑	w/t/l
padGCN	**0.158±0.047**	**0.397±0.069**	**5.323±0.422**	**0.171±0.040**	**0.791±0.064**	**0.714±0.125**	**0.742±0.050**	
MIMLBoost	N/A	N/A	N/A	N/A	N/A	N/A	N/A	
MIMLSVM	0.883±0.104•	0.583±0.148•	11.120±2.869	0.221±0.097•	0.677±0.109•	0.602±0.154•	0.648±0.026•	5/1/0
MIMLfast	0.237±0.045•	0.411±0.312	16.366±0.439•	0.807±0.074•	0.473±0.017•	0.491±0.298•	0.488±0.113•	5/1/0
DeepMIML	0.289±0.063•	0.592±0.157•	10.566±0.589•	0.193±0.016•	0.712±0.077•	0.701±0.176•	0.706±0.077•	6/0/0
M3GN	0.222±0.018•	0.499±0.143•	5.024±0.072	0.203±0.073•	0.742±0.088•	0.610±0.242	0.701±0.052•	4/2/0
MIML-GAN	0.257±0.021•	0.443±0.075•	9.646±1.032•	0.180±0.029	0.525±0.025•	0.506±0.087•	0.510±0.087•	5/1/0
MIML-LLMC	N/A	N/A	N/A	N/A	N/A	N/A	N/A	
npGCN	0.179±0.062•	0.443±0.011•	7.405±0.070•	0.209±0.010•	0.662±0.007•	0.627±0.086•	0.632±0.069•	6/0/0
esGCN	0.204±0.051•	0.489±0.025•	10.63±3.388•	0.259±0.012•	0.707±0.033•	0.688±0.046•	0.702±0.039•	6/0/0

Table 7 Performance of **padGCN** compared to nine baseline methods on MS-COCO data set

Metrics	HL ↓	OE ↓	CE ↓	RL ↓	AP ↑	AR ↑	AF_1 ↑	w/t/l
padGCN	**0.224±0.016**	**0.389±0.034**	**32.596±5.069**	**0.040±0.007**	**0.885±0.004**	**0.667±0.099**	**0.763±0.052**	
MIMLBoost	N/A	N/A	N/A	N/A	N/A	N/A	N/A	
MIMLSVM	0.903±0.080•	0.609±0.116•	45.561±2.344•	0.288±0.092•	0.414±0.049•	0.432±0.214•	0.421±0.019•	6/0/0
MIMLfast	0.295±0.004•	0.715±0.185•	65.104±0.232•	0.400±0.062•	0.367±0.003•	0.430±0.098•	0.379±0.029•	6/0/0
DeepMIML	0.021±0.007	0.582±0.021•	39.148±3.053•	0.219±0.013•	0.605±0.014•	0.616±0.045•	0.607±0.026•	5/1/0
M3GN	0.286±0.018•	0.401±0.043	23.769±0.329•	0.037±0.003	0.890±0.005	0.601±0.046•	0.773±0.024	2/4/0
MIML-GAN	0.326±0.020•	0.422±0.006•	50.843±10.549•	0.330±0.040•	0.481±0.040•	0.400±0.046•	0.421±0.057•	6/0/0
MIML-LLMC	N/A	N/A	N/A	N/A	N/A	N/A	N/A	
npGCN	0.251±0.052•	0.526±0.099•	65.774±6.124•	0.048±0.019•	0.541±0.024•	0.604±0.113•	0.550±0.013•	6/0/0
esGCN	0.253±0.001•	0.541±0.052•	47.839±13.452•	0.069±0.022•	0.628±0.027•	0.622±0.033•	0.625±0.030•	6/0/0

all 6 data sets, **padGCN** consistently outperforms most of the baselines and achieves the best performance in most cases. **padGCN** shows superior performance to all methods on all data sets in terms of nearly all the evaluation metrics. Compared with SOTA methods, **padGCN** is significantly better than it in 30 experiments, comparable to it in 12 experiments, and never worse than it.

The table presents the performance of **padGCN** compared to nine baseline methods on 6 data sets. The best performances for each data set are in bold. Symbols ↑ and ↓ indicate whether higher or lower values are preferable for the respective metrics, and • and ○ signify significant performance improvements or decrements for **padGCN** compared to the baseline methods via pairwise t-tests at a significance level of 0.05. The final row summarizes the counts of wins, ties, and losses for **padGCN**. MIMLBoost and MIML-LLMC cannot return results on some data sets in 48 hours.

Table 2 shows that **padGCN** is only comparable with MIMLfast on coverage error and is significantly better than other methods on other metrics and has a total w/t/l count of 53/1/0 on Scene data set.

Table 3 shows that **padGCN** is comparable with MIMLfast on coverage error and average precision, and DeepMIML on coverage error. **padGCN** is significantly better than other methods on other metrics and has a total w/t/l count of 51/3/0 on MIML-TEXT data set.

Table 4 shows that **padGCN** is comparable with DeepMIML on hamming loss, and M3GN on ranking loss and average precision. **padGCN** is significantly better than other methods on other metrics and has a total w/t/l count of 45/3/0 on corel5k data set.

Table 5 shows that **padGCN** is comparable with DeepMIML on coverage error, and M3GN on coverage error, ranking loss, and average precision. **padGCN** is significantly better than other methods on other metrics and has a total w/t/l count of 39/3/0 on DeliciousMIL data set.

Table 6 shows that **padGCN** is comparable with MIMLSVM on coverage error, MIMLfast on one error, M3GN on coverage error, and MIML-GAN on ranking loss. **padGCN** is significantly better than other methods on other metrics and has a total w/t/l count of 37/5/0 on FLICKR25K data set.

Table 7 shows that **padGCN** is comparable with DeepMIML on hamming loss, and M3GN on one error, ranking loss, average precision, and average F_1. **padGCN** is significantly better than other methods on other metrics and has a total w/t/l count of 37/5/0 on MS-COCO data set.

padGCN exploits both positional features and the relationships between instances and labels. In Fig. 4, we present examples from the FLICKR25K data set, illustrating the distinction between upper and lower images in terms of triggering the label *sunset* due to different distances between instances within the bag. In the upper set of images, instances representing the sun and the horizon are spatially close, prompting our method to establish relationships between them. Consequently, their aggregated features activate the label *sunset*. Conversely, in the lower set of images, instances representing the sun and instances representing the horizon are spatially distant, preventing our method from aggregating their features, and making the bag a negative sample of the label *sunset*.

4.3 Ablation study

In this subsection, ablation studies are conducted to validate the significance of various components in **padGCN**. The aim is to show the necessity of the following separate components:

- Positional features of instances. To validate the importance of positional features, we consider npGCN, which is a simplified version of our **padGCN** method. In npGCN, instances possess only semantic features, and the similarity matrix of GCN is removed while the adjacency matrix is reserved.
- Multi-label GCN projections to hyperbolic space. Another important component we investigate is the projection of sub-sub-concepts, sub-concepts, and labels, and the tripartite graph representing their relationships in hyperbolic space. To this end, we introduce esGCN, a degenerated version of **padGCN**, which lacks the projection step.

In experiments, the parameters of npGCN and esGCN are identical

Fig. 4 Provided images come from the FLICKR25K data set, with annotated instance positions in the images. Positions are represented with 2-dimension coordinates

to those in **padGCN**. The results are summarized in Tables 2–7 based on 10 runs. As the results show, **padGCN** surpasses npGCN and esGCN. Compared with them, **padGCN** benefits significantly from inter-instance distances when organizing instances in the graph structure and adjusting weights in convolutional layers, and the advantage of hyperbolic space in calculating distances in a hierarchical structure in the multi-label GCN.

4.4 Parameter sensitivity analysis

In this subsection, we conduct parameter sensitivity analyses to validate the robustness of **padGCN**. Parameter sensitivity analyses cover cluster number, latent layers, latent dimension, GCN depth, τ and β. We can see how the performance of **padGCN** changes via the fluctuation of measurement AF_1. The range of cluster number is $\{3,4,6,9,12\}$. The range of the number of latent layers is $\{1,2,3,4,5\}$. The range of latent dimension is $\{64,128,256,512,1024\}$. The range of GCN depth is $\{1,2,3,4,5\}$. The range of τ is $\{32,64,128,256,512\}$. The range of β is $\{0,0.25,0.5,0.75,1\}$.

This subsection shows how each parameter affect the model performance. For each parameter, we change its value and fix all other parameters in according parameter robustness analysis. We use AF_1 as the performance measurement and conduct experiments on data sets Scene, MIML-TEXT, corel5k, DeliciousMIL, FLICKR25K, and MS-COCO.

Figure 5 shows how AF_1 changes when cluster number, latent dimension, and GCN depth changes. Because the influence to the AF_1 of latent layers, τ and β is too little to be shown.

As shown in the figure, when parameter changes, though the value of AF_1 declines, it only slightly fluctuates. The results indicate that **padGCN** is not sensitive to all these parameters.

4.5 Time cost

Figure 6 illustrates the average runtime across 10 runs for each method. The horizontal axis displays the method names, which from left to right are **padGCN**, MIMLBoost, MIMLSVM, MIMLfast, DeepMIML, M3GN, MIML-GAN, MIML-LLMC, npGCN, and

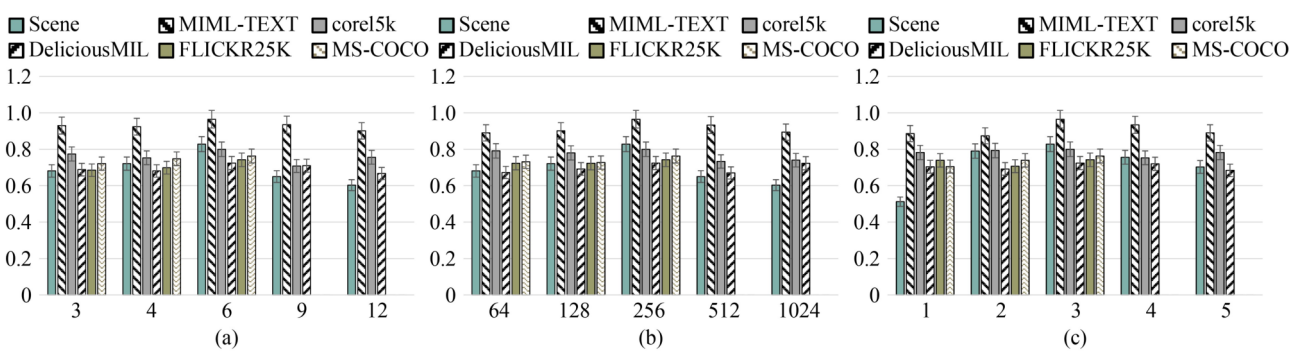

Fig. 5 Changes of AF_1 *vs.* (a) cluster number, (b) latent dimension, and (c) GCN depth. For cases when a method cannot return results within 24 hours, the corresponding cell is left blank

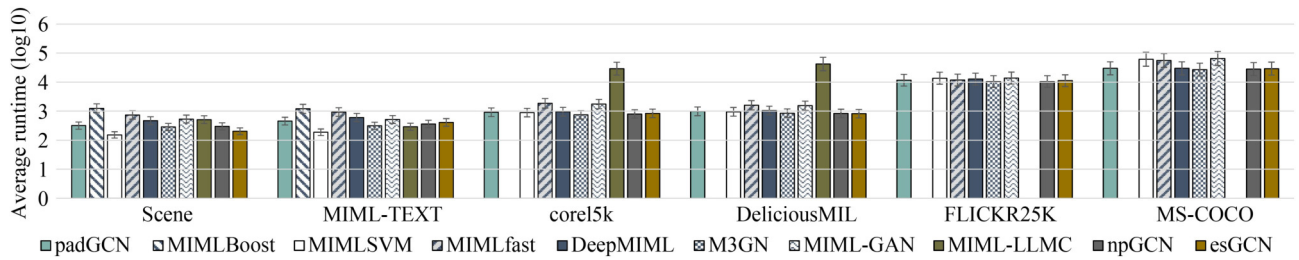

Fig. 6 Average runtime of **padGCN** and 9 baseline methods across 6 data sets. For cases when a method cannot return results within 24 hours, the corresponding cell is left blank

esGCN. The vertical axis represents the average runtime measured in seconds on a logarithmic scale.

As illustrated in Fig. 6, the methods with the shortest average runtimes include MIMLSVM, MIMLSVM, M3GN, M3GN, and M3GN, with MIMLSVM, MIMLSVM, and M3GN emerging as the top performers in terms of runtime efficiency. In data sets Scene and MIML-TEXT with smaller scales, MIMLSVM has less runtime, benefiting from SVM's efficiency on smaller data sets and developed optimizations of programs. Conversely, in data sets corel5k, FLICKR25K, and MS-COCO with larger scales, M3GN outperforms others due to its utilization of GAT networks and developed optimizations of programs. It is worth noting that in **padGCN**, the computation of similarity matrices is a time-intensive process. For small-scale data sets, deep learning-based methods tend to be slower than those relying on SVM. However, for large-scale data sets, the average runtimes of npGCN, a simplified version of **padGCN** that excludes similarity matrices, closely match those of M3GN. In general, **padGCN** cost equivalent runtime with SOTA methods.

5 Conclusion

In summary, this paper introduces **padGCN**, a novel method for MIML learning that employs position-aware doubly GCNs. Our contributions can be summarized as follows: 1) We propose an innovative bag generation method that consists of instances' positional features which are neglected in the past. 2) We propose a weighted GCN approach for aggregating instances' features with the similarity matrix calculated based on both their positional and semantic features, innovatively introducing positional features into the multi-instance relationship exploitation step. 3) We learn relationships between sub-sub-concepts, sub-concepts, and labels using a tripartite GCN in hyperbolic space. Experiments validate the superiority, robustness, and efficiency of our method.

Acknowledgements

This work was supported by the National Science and Technology Major Project (Grant No. 2022ZD0114803), the Natural Science Foundation of Wuhan (Grant No. 2023010201020229), and the Major Program (JD) of Hubei Province (Grant No. 2023BAA024).

Competing interests

The authors declare that they have no competing interests or financial conflicts to disclose.

References

[1] Zhou Z H, Zhang M L. Multi-instance multi-label learning with application to scene classification. In: Proceedings of the 20th International Conference on Neural Information Processing Systems. 2006, 1609−1616

[2] Chen Z, Chi Z, Fu H, Feng D. Multi-instance multi-label image classification: a neural approach. Neurocomputing, 2013, 99: 298−306

[3] Zha Z J, Hua X S, Mei T, Wang J, Qi G J, Wang Z. Joint multi-label multi-instance learning for image classification. In: Proceedings of 2018 IEEE Conference on Computer Vision and Pattern Recognition. 2008, 1−8

[4] Zhang Q, Goldman S A, Yu W, Fritts J. Content-based image retrieval using multiple-instance learning. In: Proceedings of the 19th International Conference on Machine Learning. 2002, 682−689

[5] Yan K, Li Z, Zhang C. A new multi-instance multi-label learning approach for image and text classification. Multimedia Tools and Applications, 2016, 75(13): 7875−7890

[6] Surdeanu M, Tibshirani J, Nallapati R, Manning C D. Multi-instance multi-label learning for relation extraction. In: Proceedings of 2012 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning. 2012, 455−465

[7] Dietterich T G, Lathrop R H, Lozano-Pérez T. Solving the multiple instance problem with axis-parallel rectangles. Artificial Intelligence, 1997, 89(1−2): 31−71

[8] Ding X, Li B, Xiong W, Guo W, Hu W, Wang B. Multi-instance multi-label learning combining hierarchical context and its application to image annotation. IEEE Transactions on Multimedia, 2016, 18(8): 1616−1627

[9] Wang H, Dong L, Sun M. Local feature aggregation algorithm based on graph convolutional network. Frontiers of Computer Science, 2022, 16(3): 163309

[10] Li Y X, Ji S, Kumar S, Ye J, Zhou Z H. Drosophila gene expression pattern annotation through multi-instance multi-label learning. IEEE/ACM Transactions on Computational Biology and Bioinformatics, 2012, 9(1): 98−112

[11] Xu Y, Min H, Song H, Wu Q. Multi-instance multi-label distance metric learning for genome-wide protein function prediction. Computational Biology and Chemistry, 2016, 63: 30−40

[12] Xiao L, Chen B L, Huang X, Liu H F, Jing L P, Yu J. Multi-label text classification method based on label semantic information. Journal of Software, 2020, 31(4): 1079−1089

[13] Ma H, Lu N, Mei J, Guan T, Zhang Y, Geng X. Label distribution learning for scene text detection. Frontiers of Computer Science, 2023, 17(6): 176339

[14] Ren Y, Xu N, Ling M, Geng X. Label distribution for multimodal machine learning. Frontiers of Computer Science, 2022, 16(1): 161306

[15] Dong X, Luo T, Fan R, Zhuge W, Hou C. Active label distribution

learning via kernel maximum mean discrepancy. Frontiers of Computer Science, 2023, 17(4): 174327
[16] Tang W, Zhang W, Zhang M L. Multi-instance partial-label learning: towards exploiting dual inexact supervision. Science China Information Sciences, 2024, 67(3): 132103
[17] Feng J, Zhou Z H. Deep MIML network. In: Proceedings of the 31st AAAI Conference on Artificial Intelligence. 2017, 1884−1890
[18] Huang S J, Gao W, Zhou Z H. Fast multi-instance multi-label learning. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2019, 41(11): 2614−2627
[19] Xing Y, Yu G, Domeniconi C, Wang J, Zhang Z L, Guo M. Multi-view multi-instance multi-label learning based on collaborative matrix factorization. In: Proceedings of the 33rd AAAI Conference on Artificial Intelligence. 2019, 5508−5515
[20] Hang C, Wang W, Zhan D C. Multi-modal multi-instance multi-label learning with graph convolutional network. In: Proceedings of 2021 International Joint Conference on Neural Networks. 2021, 1−8
[21] Yang M, Tang W T, Min F. Multi-instance multi-label learning based on parallel attention and local label manifold correlation. In: Proceedings of 2022 IEEE 9th International Conference on Data Science and Advanced Analytics. 2022, 1−10
[22] Zhang M L, Wang Z J. MIMLRBF: RBF neural networks for multi-instance multi-label learning. Neurocomputing, 2009, 72(16−18): 3951−3956
[23] Li Y F, Hu J H, Jiang Y, Zhou Z H. Towards discovering what patterns trigger what labels. In: Proceedings of the 26th AAAI Conference on Artificial Intelligence. 2012, 1012−1018
[24] Tu M, Huang J, He X, Zhou B. Multiple instance learning with graph neural networks. 2019, arXiv preprint arXiv: 1906.04881
[25] Zhang M L. A k-nearest neighbor based multi-instance multi-label learning algorithm. In: Proceedings of the 22nd IEEE International Conference on Tools with Artificial Intelligence. 2010, 207−212
[26] Pan Z, Wang B, Zhang R, Wang S, Li Y. MIML-GAN: a GAN-based algorithm for multi-instance multi-label learning on overlapping signal waveform recognition. IEEE Transactions on Signal Processing, 2023, 71(1): 859−872
[27] Zhang M L, Zhou Z H. M3MIML: a maximum margin method for multi-instance multi-label learning. In: Proceedings of the 18th IEEE International Conference on Data Mining. 2008, 688−697
[28] Ghaseminezhad M H, Karami A. A novel self-organizing map (SOM) neural network for discrete groups of data clustering. Applied Soft Computing, 2011, 11(4): 3771−3778
[29] Andrews S, Tsochantaridis I, Hofmann T. Support vector machines for multiple-instance learning. In: Proceedings of the 16th International Conference on Neural Information Processing Systems. 2002, 577−584
[30] Devlin J, Chang M W, Lee K, Toutanova K. BERT: pre-training of deep bidirectional transformers for language understanding. In: Proceedings of 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers). 2019, 4171−4186
[31] Chen T, Kornblith S, Norouzi M, Hinton G. A simple framework for contrastive learning of visual representations. In: Proceedings of the 37th International Conference on Machine Learning. 2020, 149
[32] Xiao S, Bai T, Cui X, Wu B, Meng X, Wang B. A graph-based contrastive learning framework for medicare insurance fraud detection. Frontiers of Computer Science, 2023, 17(2): 172341
[33] Ma X, Li Z, Song G, Shi C. Learning discrete adaptive receptive fields for graph convolutional networks. Science China Information Sciences, 2023, 66(12): 222101
[34] Zhou L, Wang T, Qu H, Huang L, Liu Y. A weighted GCN with logical adjacency matrix for relation extraction. In: Proceedings of the 24th European Conference on Artificial Intelligence. 2020, 2314−2321
[35] Chinchor N, Sundheim B. MUC-5 evaluation metrics. In: Proceedings of the 5th Conference on Message Understanding Conference. 1993, 69−78
[36] Chami I, Ying R, Re C, Leskovec J. Hyperbolic graph convolutional neural networks. In: Proceedings of the 33rd International Conference on Neural Information Processing Systems. 2019, 438
[37] Kingma D P, Ba J. Adam: a method for stochastic optimization. In: Proceedings of the 3rd International Conference on Learning Representations. 2014, 1−15
[38] Maron O, Ratan A L. Multiple-instance learning for natural scene classification. In: Proceedings of the 15th International Conference on Machine Learning. 1998, 341−349
[39] Sebastiani F. Machine learning in automated text categorization. ACM Computing Surveys (CSUR), 2002, 34(1): 1−47
[40] Duygulu P, Barnard K, de Freitas J F G, Forsyth D A. Object recognition as machine translation: learning a lexicon for a fixed image vocabulary. In: Proceedings of the 7th European Conference on Computer Vision. 2002, 97−112
[41] Huiskes M J, Lew M S. The MIR flickr retrieval evaluation. In: Proceedings of the 1st ACM International Conference on Multimedia Information Retrieval. 2008, 39−43
[42] Lin T Y, Maire M, Belongie S, Hays J, Perona P, Ramanan D, Dollár P, Zitnick C L. Microsoft coco: common objects in context. In: Proceedings of the 13th European Conference on Computer Vision. 2014, 740−755
[43] Xu X, Frank E. Logistic regression and boosting for labeled bags of instances. In: Proceedings of the 8th Pacific-Asia Conference on Advances in Knowledge Discovery and Data Mining. 2004, 272−281
[44] Ren S, He K, Girshick R, Sun J. Faster R-CNN: towards real-time object detection with region proposal networks. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2017, 39(6): 1137−1149

Zhi LI is a Master's candidate at Huazhong University of Science and Technology, China, specializing in Computer Science and graduated in 2024. His research focuses on multi-instance multi-label learning and graph neural networks, with a particular emphasis on integrating positional features and hierarchical label dependencies. As a key contributor to the padGCN framework, he pioneered methods to model spatial relationships in image/text classification tasks, achieving state-of-the-art performance on benchmark datasets (e.g., MS-COCO).

Teng ZHANG is an associate professor of the College of Computer Science and Technology, Huazhong University of Science and Technology, China. He received his PhD degree from the Department of Computer Science and Technology, Nanjing University, China in 2019. His research interests include machine learning, data mining, and optimization.

Yilin WANG received his Master's degree in Computer Science and Technology from Huazhong University of Science and Technology, China in 2022. He is currently working in Bytedance Inc. His research interests mainly include margin based methods and recommender system.

Caiwu JIANG received his bachelor's degree in Intelligent Science and Technology from Sun Yat-sen University, China and obtained a master's degree from Huazhong University of Science and Technology, China. He conducted research under the supervision of Associate Professor Teng Zhang, specializing in kernel methods. His work focused on developing kernel-based machine learning frameworks, with particular emphasis on theoretical analysis of reproducing kernel Hilbert spaces and efficient optimization algorithms.

Xuanhua SHI is a professor in National Engineering Research Center for Big Data Technology and System/Services Computing Technology and System Lab, Huazhong University of Science and Technology, China. He received the PhD degree in computer engineering from Huazhong University of Science and Technology, China in 2005. From 2006, he worked as an INRIA Post-Doc in PARIS team at Rennes for one year. His current research interests focus on the cloud computing and big data processing. He published over 100 peer-reviewed publications, received research support from a variety of governmental and industrial organizations, such as the National Natural Science Foundation of China, the Ministry of Science and Technology of China, the Ministry of Education of China, the European Union, Alibaba, ByteDance, and Intel. Shi is a senior member of IEEE and CCF.

Hai JIN received his PhD degree in computer engineering from Huazhong University of Science and Technology, China in 1994. He received German Academic Exchange Service fellowship to visit the Technical University of Chemnitz, Germany in 1996. He worked at the University of Hong Kong, China between 1998 and 2000, and as a visiting scholar at the University of Southern California, USA between 1999 and 2000. He received the Excellent Youth Award from the National Natural Science Foundation of China in 2001. He is a Cheung Kung Scholars chair professor of Computer Science and Engineering of Huazhong University of Science and Technology, the chief scientist of ChinaGrid, the largest grid computing project in China, and the chief scientist of National 973 Basic Research Program Project of Virtualization Technology of Computing System, and Cloud Security. He has coauthored 22 books and published over 800 research papers. His research interests include computer architecture, virtualization technology, cluster computing and cloud computing, peer-to-peer computing, network storage, and network security. He is a fellow of the CCF, a fellow of the IEEE, and a member of the ACM.